EXPERIMENT -6

AIM- Create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

OBJECTIVES

1. To understand the concept of stored procedures in SQL.
2. To create a stored procedure for transferring an employee between departments.
3. To implement input validation and error handling using SIGNAL.
4. To understand SQL triggers and their execution timing.
5. To create a trigger for validating employee salaries.
6. To create an audit table for maintaining salary-change history.
7. To create an audit trigger for recording salary modifications.
8. To test valid and invalid inputs using different edge cases.

THEORY

Stored Procedure

A stored procedure is a named collection of SQL statements stored in the database and executed when called.

Stored procedures are useful for:

- Reusing frequently performed operations.
- Encapsulating business logic.
- Performing validation.
- Handling transactions.

Parameterized Stored Procedure

A stored procedure can accept parameters from the user or application.

For example: transfer_employee(emp_id, new_dept_id)

Here:

- `emp_id` identifies the employee.
- `new_dept_id` identifies the destination department.

Error Handling

Error handling allows a procedure to respond appropriately when an invalid condition occurs.

For example, an error can be generated when:

- Employee does not exist.
- Department does not exist.
- Employee is already in the requested department.
- Invalid input is supplied.

Transaction

A transaction groups multiple database operations into a single logical unit.

The main transaction commands are:

- START TRANSACTION
- COMMIT
- ROLLBACK

Trigger

A trigger is a database object that automatically executes when a specified event occurs on a table.

Triggers can execute:

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER UPDATE
- BEFORE DELETE
- AFTER DELETE

Salary Validation Trigger

A salary validation trigger can ensure that invalid salary values are not inserted or updated.

For example, a salary can be required to be greater than zero.

A BEFORE INSERT trigger validates newly inserted employees, while a BEFORE UPDATE trigger validates salary changes.

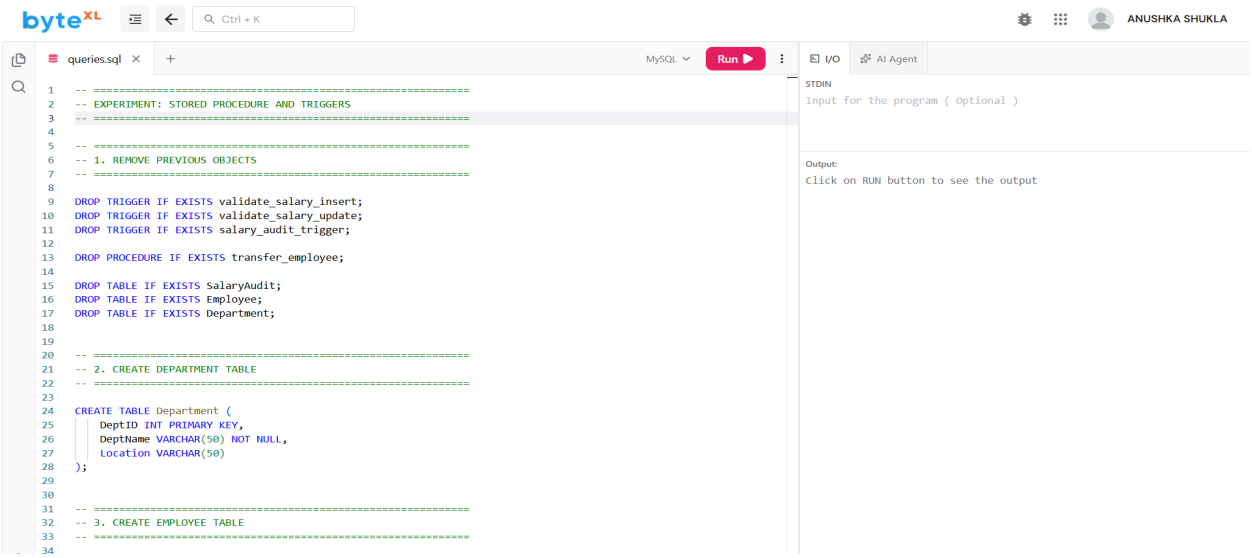
Audit Logging

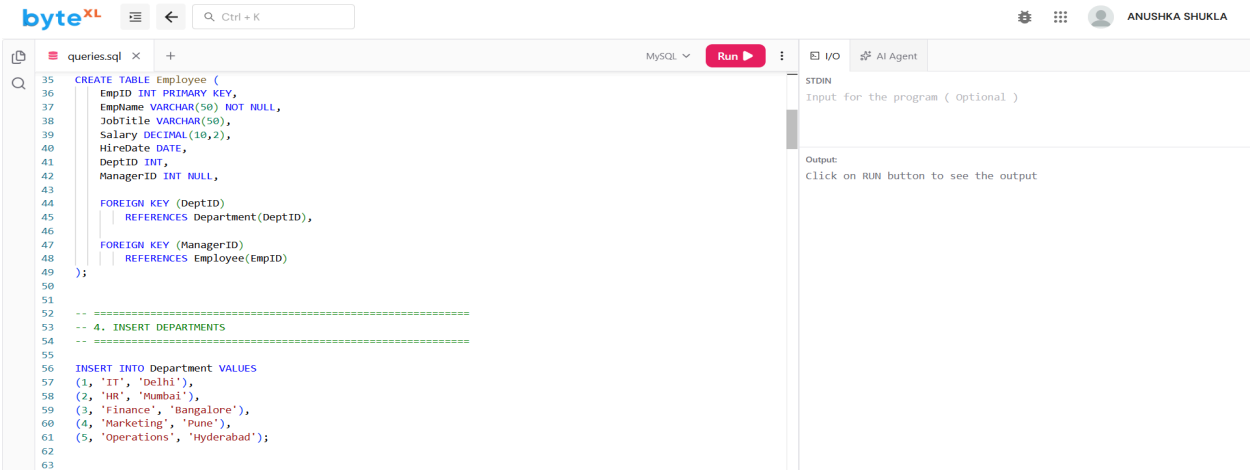
Audit logging records important changes made to database records.

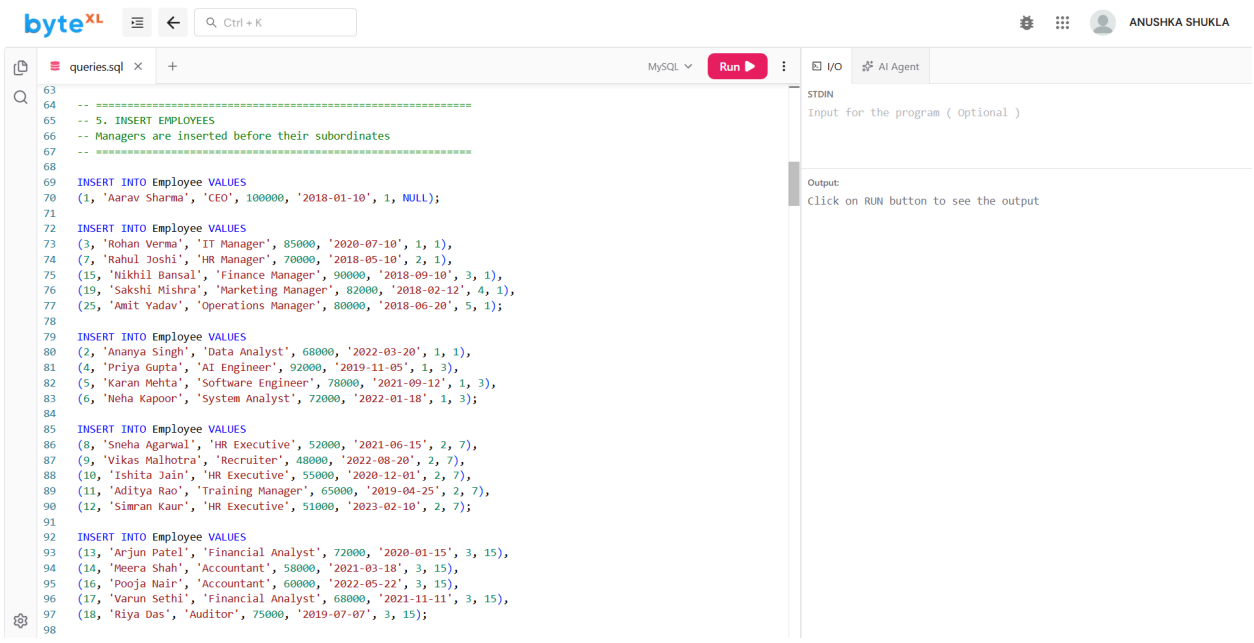
For salary changes, an audit table can store:

- Employee ID
- Old salary
- New salary
- Date and time of change
- Type of operation

SOURCE CODE








```sql
-- ============================================================
-- 5. INSERT EMPLOYEES
-- Managers are inserted before their subordinates
-- ============================================================

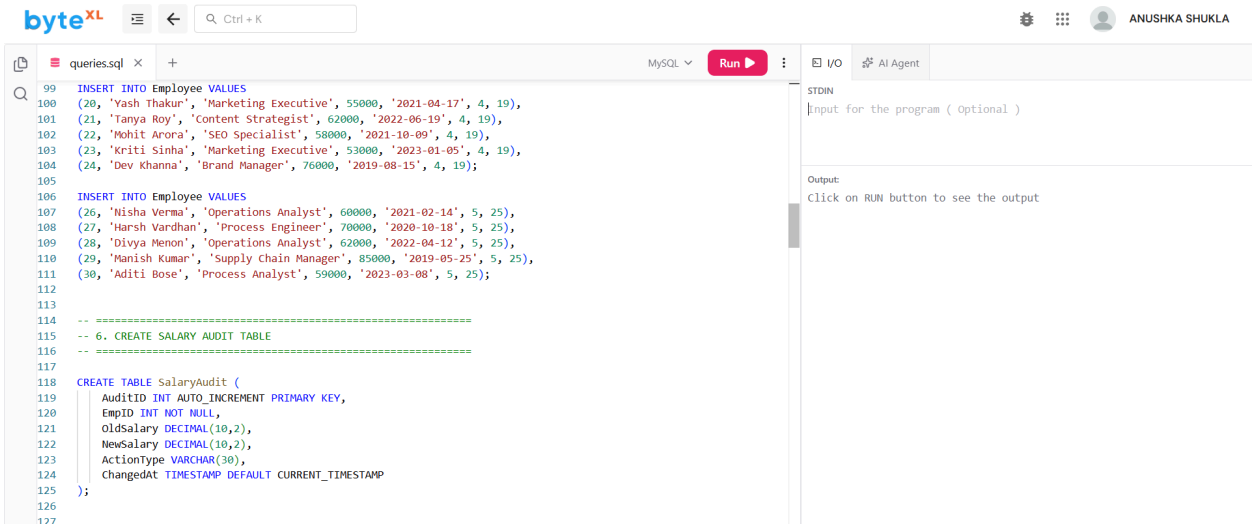
INSERT INTO Employee VALUES
(1, 'Aarav Sharma', 'CEO', 100000, '2018-01-10', 1, NULL);

INSERT INTO Employee VALUES
(3, 'Rohan Verma', 'IT Manager', 85000, '2020-07-10', 1, 1),
(7, 'Rahul Joshi', 'HR Manager', 70000, '2018-05-10', 2, 1),
(15, 'Nikhil Bansal', 'Finance Manager', 90000, '2018-09-10', 3, 1),
(19, 'Sakshi Mishra', 'Marketing Manager', 82000, '2018-02-12', 4, 1),
(25, 'Amit Yadav', 'Operations Manager', 80000, '2018-06-20', 5, 1);

INSERT INTO Employee VALUES
(2, 'Ananya Singh', 'Data Analyst', 68000, '2022-03-20', 1, 1),
(4, 'Priya Gupta', 'AI Engineer', 92000, '2019-11-05', 1, 3),
(5, 'Karan Mehta', 'Software Engineer', 78000, '2021-09-12', 1, 3),
(6, 'Neha Kapoor', 'System Analyst', 72000, '2022-01-18', 1, 3);

INSERT INTO Employee VALUES
(8, 'Sneha Agarwal', 'HR Executive', 52000, '2021-06-15', 2, 7),
(9, 'Vikas Malhotra', 'Recruiter', 48000, '2022-08-20', 2, 7),
(10, 'Ishita Jain', 'HR Executive', 55000, '2020-12-01', 2, 7),
(11, 'Aditya Rao', 'Training Manager', 65000, '2019-04-25', 2, 7),
(12, 'Simran Kaur', 'HR Executive', 51000, '2023-02-10', 2, 7);

INSERT INTO Employee VALUES
(13, 'Arjun Patel', 'Financial Analyst', 72000, '2020-01-15', 3, 15),
(14, 'Meera Shah', 'Accountant', 58000, '2021-03-18', 3, 15),
(16, 'Pooja Nair', 'Accountant', 60000, '2022-05-22', 3, 15),
(17, 'Varun Sethi', 'Financial Analyst', 68000, '2021-11-11', 3, 15),
(18, 'Riya Das', 'Auditor', 75000, '2019-07-07', 3, 15);
```

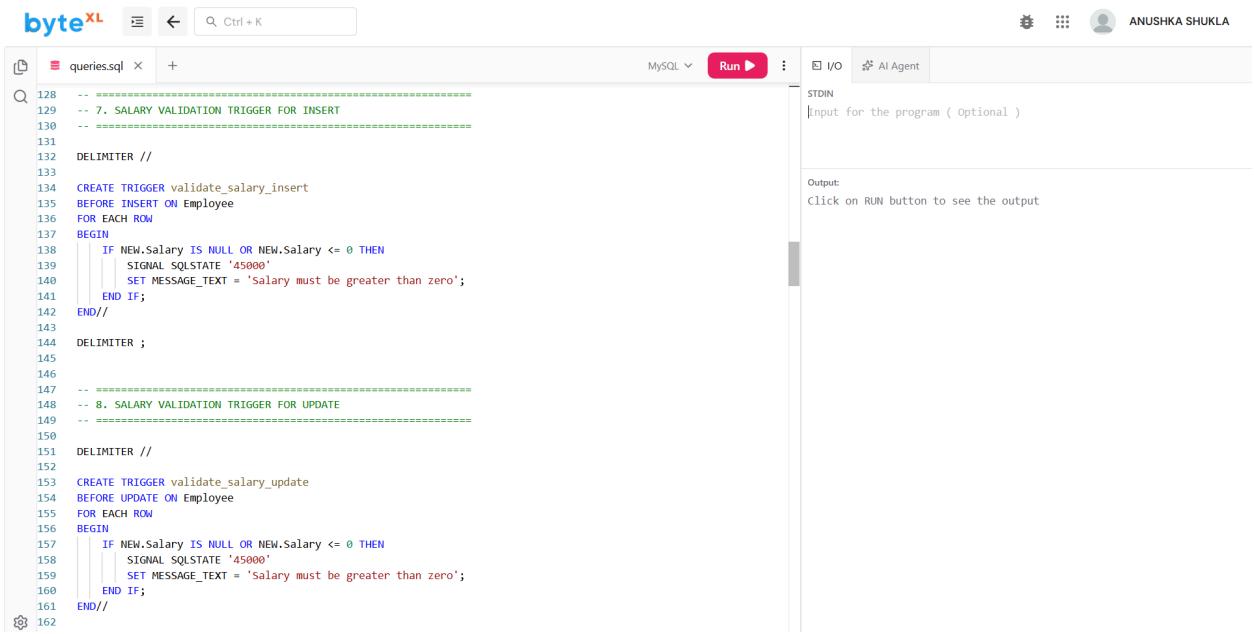



```sql
INSERT INTO Employee VALUES
(20, 'Yash Thakur', 'Marketing Executive', 55000, '2021-04-17', 4, 19),
(21, 'Tanya Roy', 'Content Strategist', 62000, '2022-06-19', 4, 19),
(22, 'Mohit Arora', 'SEO Specialist', 58000, '2021-10-09', 4, 19),
(23, 'Kriti Sinha', 'Marketing Executive', 53000, '2023-01-05', 4, 19),
(24, 'Dev Khanna', 'Brand Manager', 76000, '2019-08-15', 4, 19);

INSERT INTO Employee VALUES
(26, 'Nisha Verma', 'Operations Analyst', 60000, '2021-02-14', 5, 25),
(27, 'Harsh Vardhan', 'Process Engineer', 70000, '2020-10-18', 5, 25),
(28, 'Divya Menon', 'Operations Analyst', 62000, '2022-04-12', 5, 25),
(29, 'Manish Kumar', 'Supply Chain Manager', 85000, '2019-05-25', 5, 25),
(30, 'Aditi Bose', 'Process Analyst', 59000, '2023-03-08', 5, 25);


-- ============================================================
-- 6. CREATE SALARY AUDIT TABLE
-- ============================================================

CREATE TABLE SalaryAudit (
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    EmpID INT NOT NULL,
    OldSalary DECIMAL(10,2),
    NewSalary DECIMAL(10,2),
    ActionType VARCHAR(30),
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```




```sql
-- ============================================================
-- 7. SALARY VALIDATION TRIGGER FOR INSERT
-- ============================================================

DELIMITER //

CREATE TRIGGER validate_salary_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary IS NULL OR NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END//

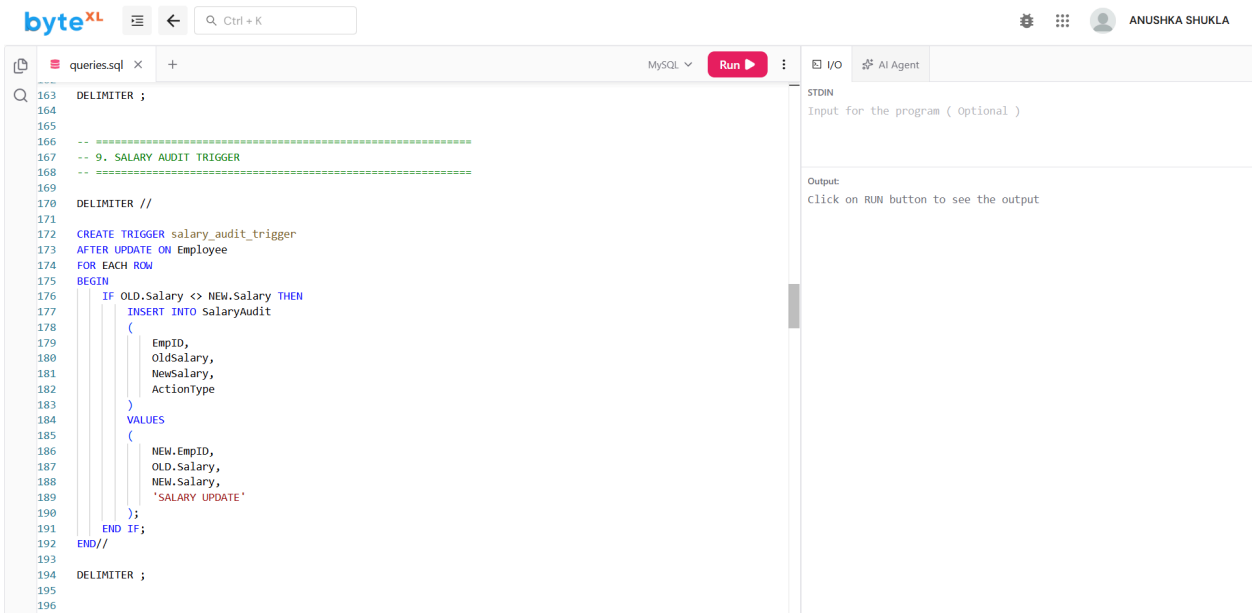
DELIMITER ;


-- ============================================================
-- 8. SALARY VALIDATION TRIGGER FOR UPDATE
-- ============================================================

DELIMITER //

CREATE TRIGGER validate_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary IS NULL OR NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END//

```




```sql
DELIMITER ;

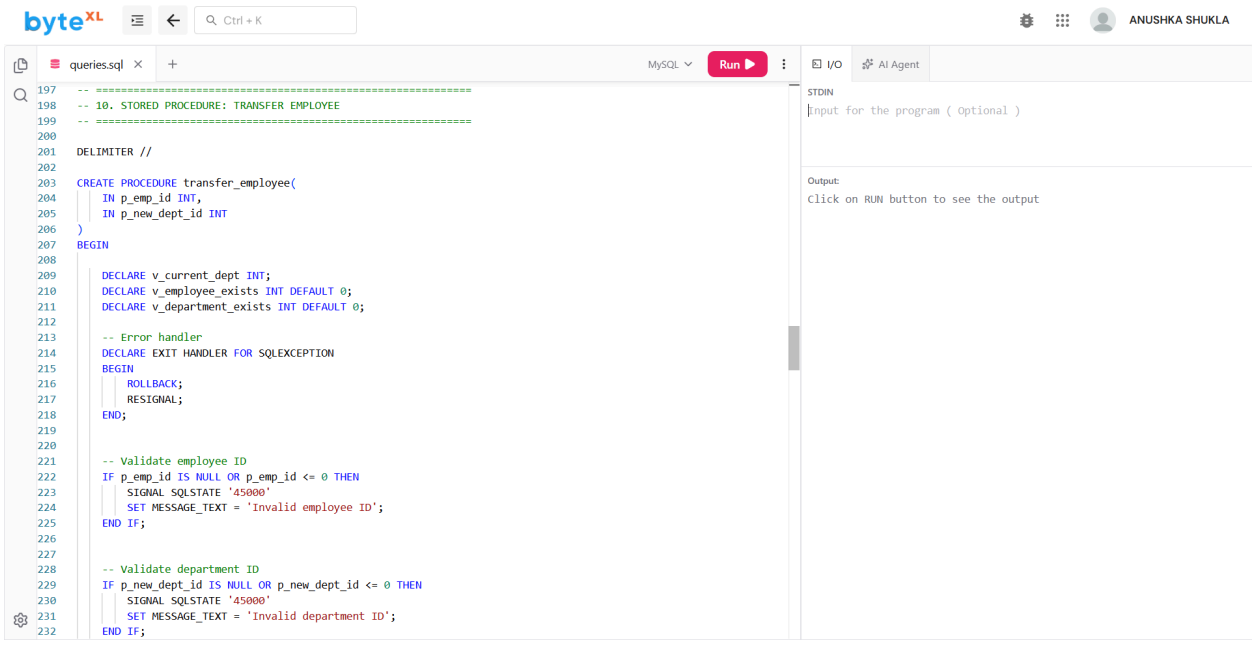

-- ============================================================
-- 9. SALARY AUDIT TRIGGER
-- ============================================================

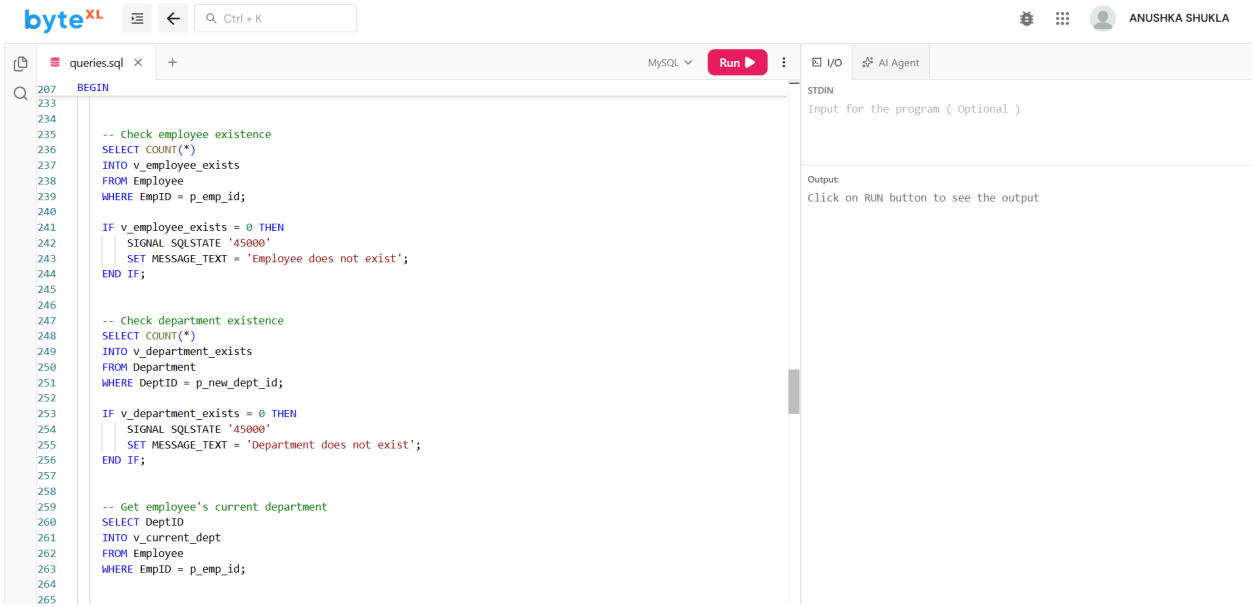
DELIMITER //

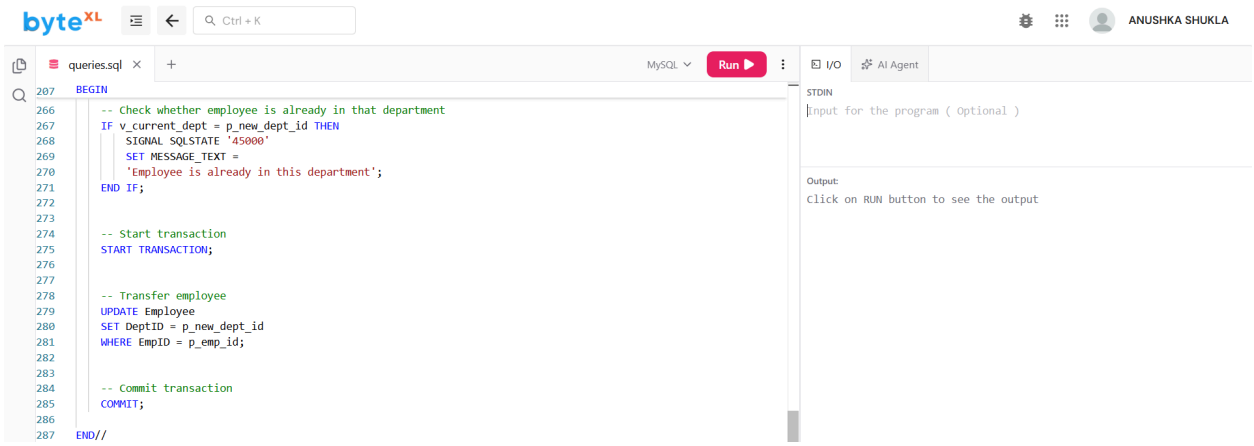
CREATE TRIGGER salary_audit_trigger
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO SalaryAudit
        (
            EmpID,
            OldSalary,
            NewSalary,
            ActionType
        )
        VALUES
        (
            NEW.EmpID,
            OLD.Salary,
            NEW.Salary,
            'SALARY UPDATE'
        );
    END IF;
END//

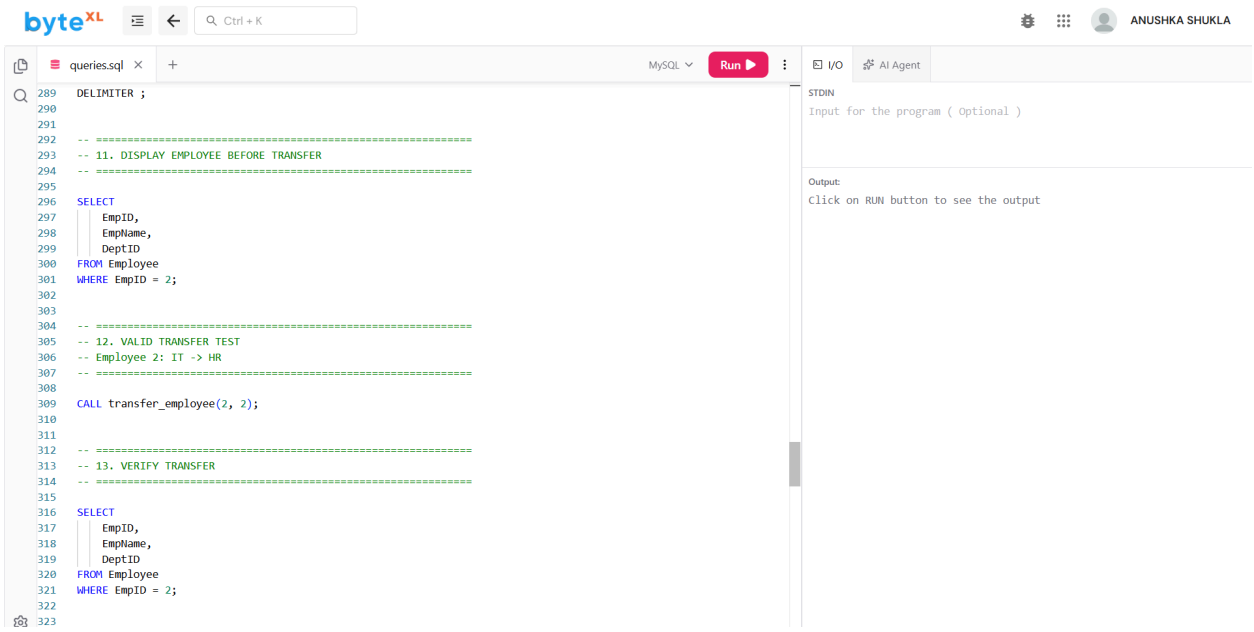
DELIMITER ;

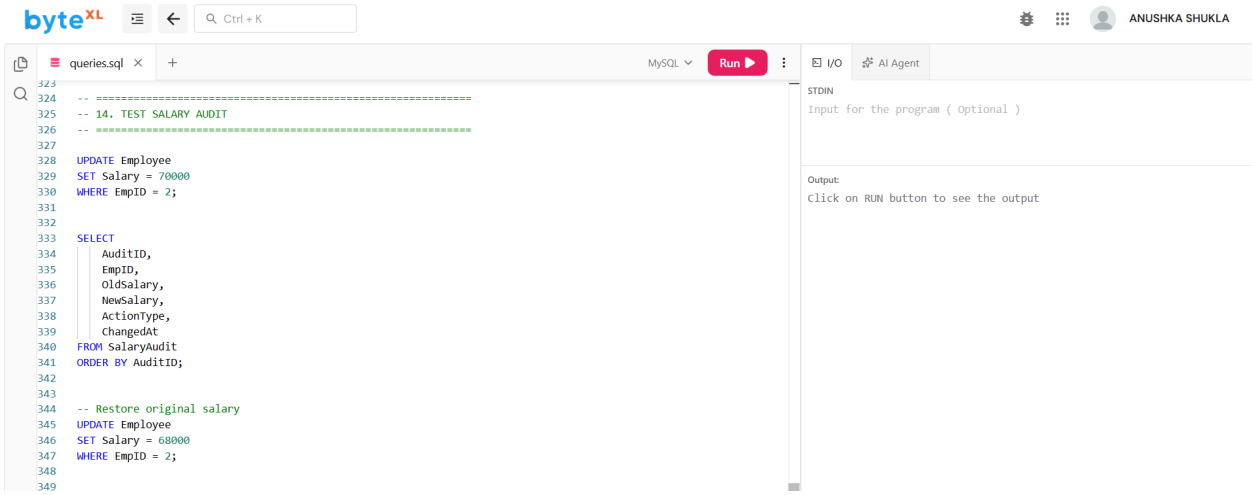
```

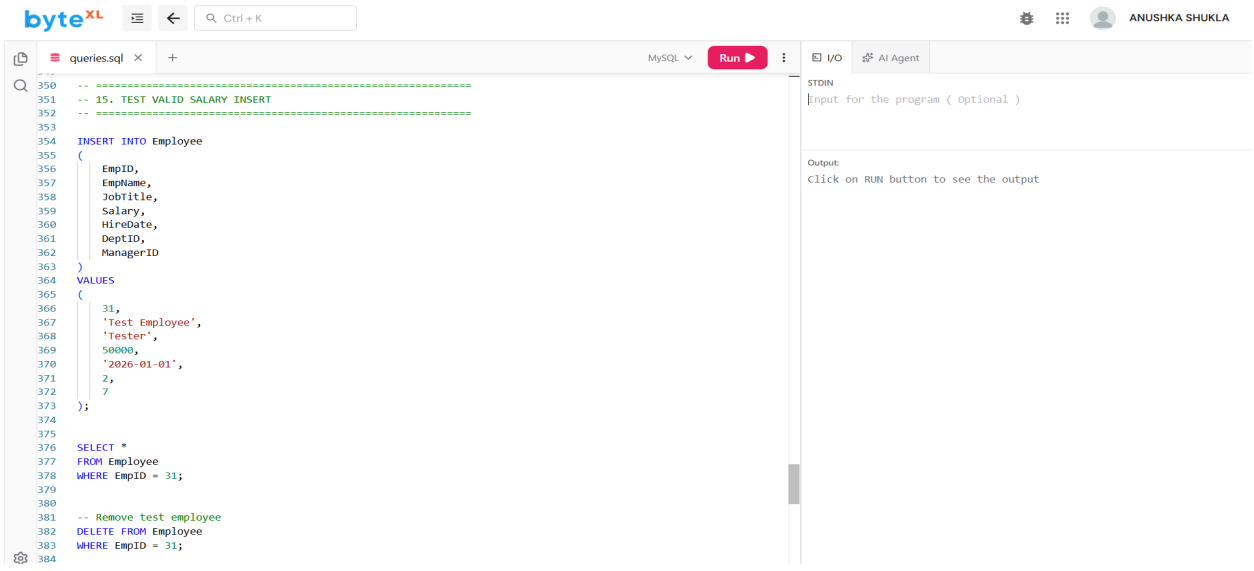

byteXL
Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==
-- 10. STORED PROCEDURE: TRANSFER EMPLOYEE
-- ==
DELIMITER //
CREATE PROCEDURE transfer_employee(
IN p_emp_id INT,
IN p_new_dept_id INT
)
BEGIN
DECLARE v_current_dept INT;
DECLARE v_employee_exists INT DEFAULT 0;
DECLARE v_department_exists INT DEFAULT 0;
-- Error handler
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
ROLLBACK;
RESIGNAL;
END;
-- Validate employee ID
IF p_emp_id IS NULL OR p_emp_id <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Invalid employee ID';
END IF;
-- Validate department ID
IF p_new_dept_id IS NULL OR p_new_dept_id <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Invalid department ID';
END IF;


byteXL
Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
BEGIN
-- Check employee existence
SELECT COUNT(*)
INTO v_employee_exists
FROM Employee
WHERE EmpID = p_emp_id;
IF v_employee_exists = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee does not exist';
END IF;
-- Check department existence
SELECT COUNT(*)
INTO v_department_exists
FROM Department
WHERE DeptID = p_new_dept_id;
IF v_department_exists = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Department does not exist';
END IF;
-- Get employee's current department
SELECT DeptID
INTO v_current_dept
FROM Employee
WHERE EmpID = p_emp_id;


byteXL
Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
207 BEGIN
266 -- Check whether employee is already in that department
267 IF v_current_dept = p_new_dept_id THEN
268 SIGNAL SQLSTATE '45000'
269 SET MESSAGE_TEXT =
270 'Employee is already in this department';
271 END IF;
272
273
274 -- Start transaction
275 START TRANSACTION;
276
277
278 -- Transfer employee
279 UPDATE Employee
280 SET DeptID = p_new_dept_id
281 WHERE EmpID = p_emp_id;
282
283
284 -- Commit transaction
285 COMMIT;
286
287 END//


byteXL
Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
289 DELIMITER ;
290
291
292 -- ==
293 -- 11. DISPLAY EMPLOYEE BEFORE TRANSFER
294 -- ==
295
296 SELECT
297 EmpID,
298 EmpName,
299 DeptID
300 FROM Employee
301 WHERE EmpID = 2;
302
303
304 -- ==
305 -- 12. VALID TRANSFER TEST
306 -- Employee 2: IT -> HR
307 -- ==
308
309 CALL transfer_employee(2, 2);
310
311
312 -- ==
313 -- 13. VERIFY TRANSFER
314 -- ==
315
316 SELECT
317 EmpID,
318 EmpName,
319 DeptID
320 FROM Employee
321 WHERE EmpID = 2;
322
323


byteXL
Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
324 -- ==
325 -- 14. TEST SALARY AUDIT
326 -- ==
327
328 UPDATE Employee
329 SET Salary = 70000
330 WHERE EmpID = 2;
331
332
333 SELECT
334 AuditID,
335 EmpID,
336 OldSalary,
337 NewSalary,
338 ActionType,
339 ChangedAt
340 FROM SalaryAudit
341 ORDER BY AuditID;
342
343
344 -- Restore original salary
345 UPDATE Employee
346 SET Salary = 68000
347 WHERE EmpID = 2;
348
349

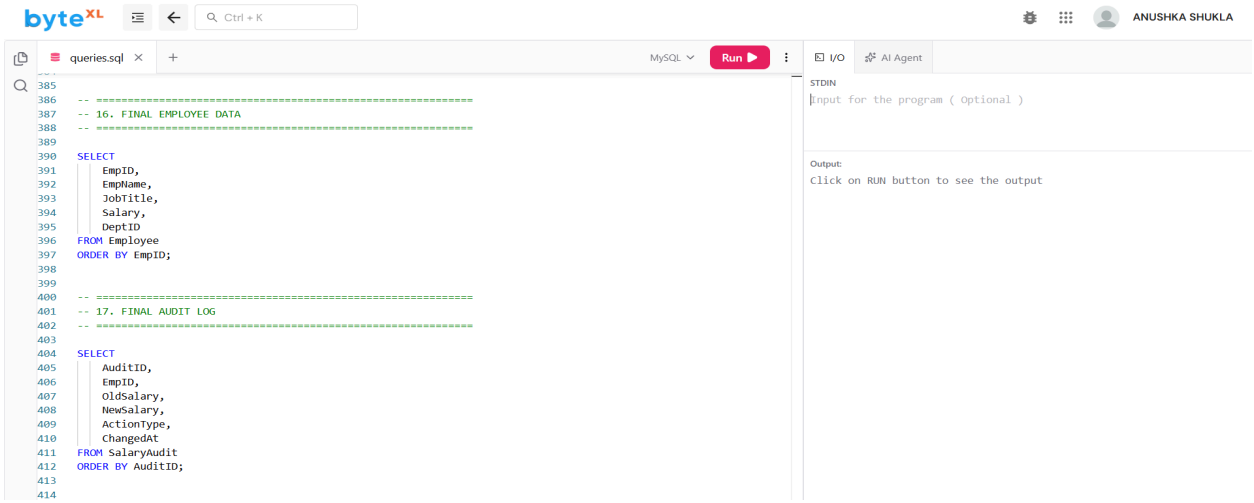



```sql
-- ============================================================
-- 15. TEST VALID SALARY INSERT
-- ============================================================

INSERT INTO Employee
(
    EmpID,
    EmpName,
    JobTitle,
    Salary,
    HireDate,
    DeptID,
    ManagerID
)
VALUES
(
    31,
    'Test Employee',
    'Tester',
    50000,
    '2026-01-01',
    2,
    7
);


SELECT *
FROM Employee
WHERE EmpID = 31;


-- Remove test employee
DELETE FROM Employee
WHERE EmpID = 31;
```




```sql

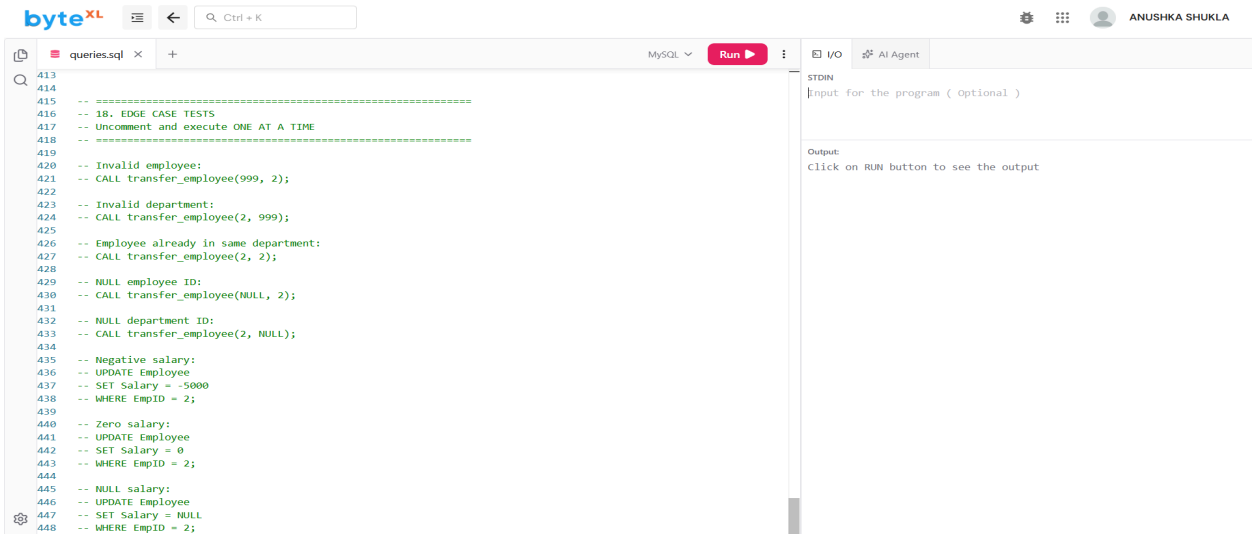
-- ============================================================
-- 16. FINAL EMPLOYEE DATA
-- ============================================================

SELECT
    EmpID,
    EmpName,
    JobTitle,
    Salary,
    DeptID
FROM Employee
ORDER BY EmpID;


-- ============================================================
-- 17. FINAL AUDIT LOG
-- ============================================================

SELECT
    AuditID,
    EmpID,
    OldSalary,
    NewSalary,
    ActionType,
    ChangedAt
FROM SalaryAudit
ORDER BY AuditID;


```




```sql


-- ============================================================
-- 18. EDGE CASE TESTS
-- Uncomment and execute ONE AT A TIME
-- ============================================================

-- Invalid employee:
-- CALL transfer_employee(999, 2);

-- Invalid department:
-- CALL transfer_employee(2, 999);

-- Employee already in same department:
-- CALL transfer_employee(2, 2);

-- NULL employee ID:
-- CALL transfer_employee(NULL, 2);

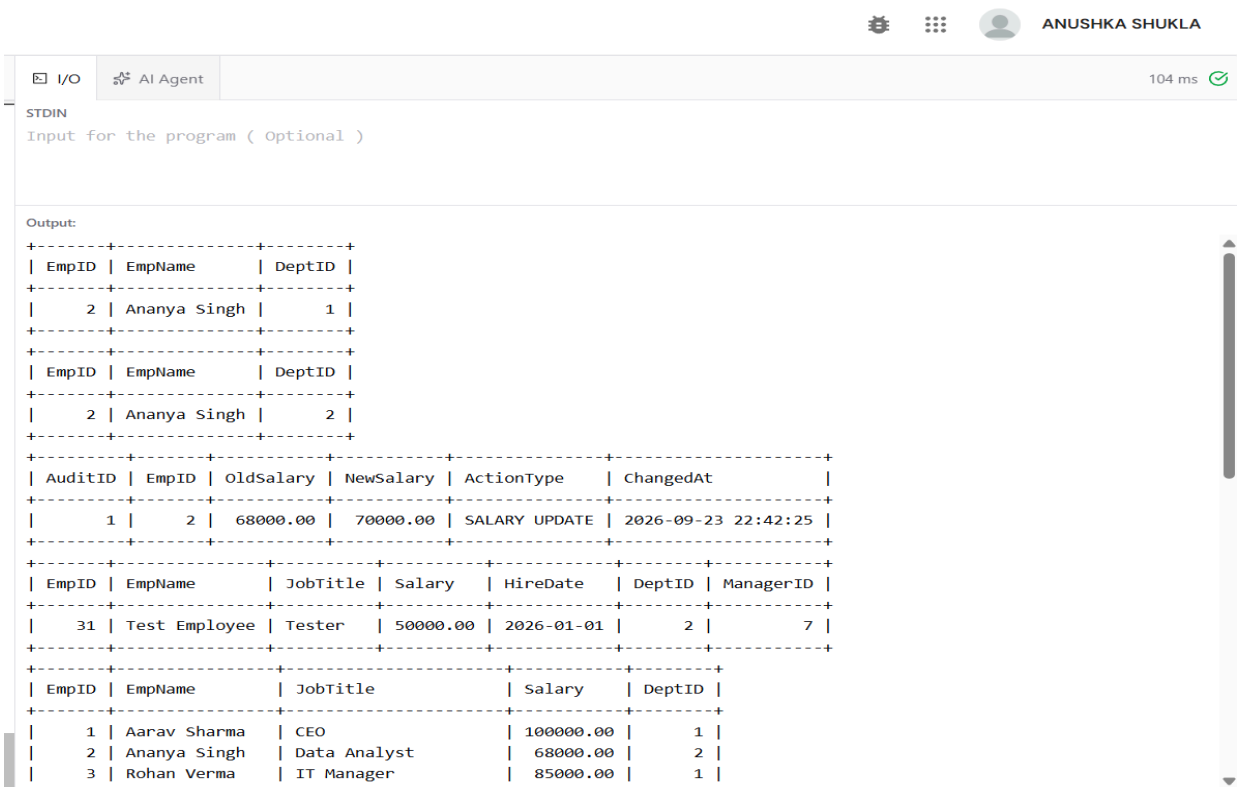
-- NULL department ID:
-- CALL transfer_employee(2, NULL);

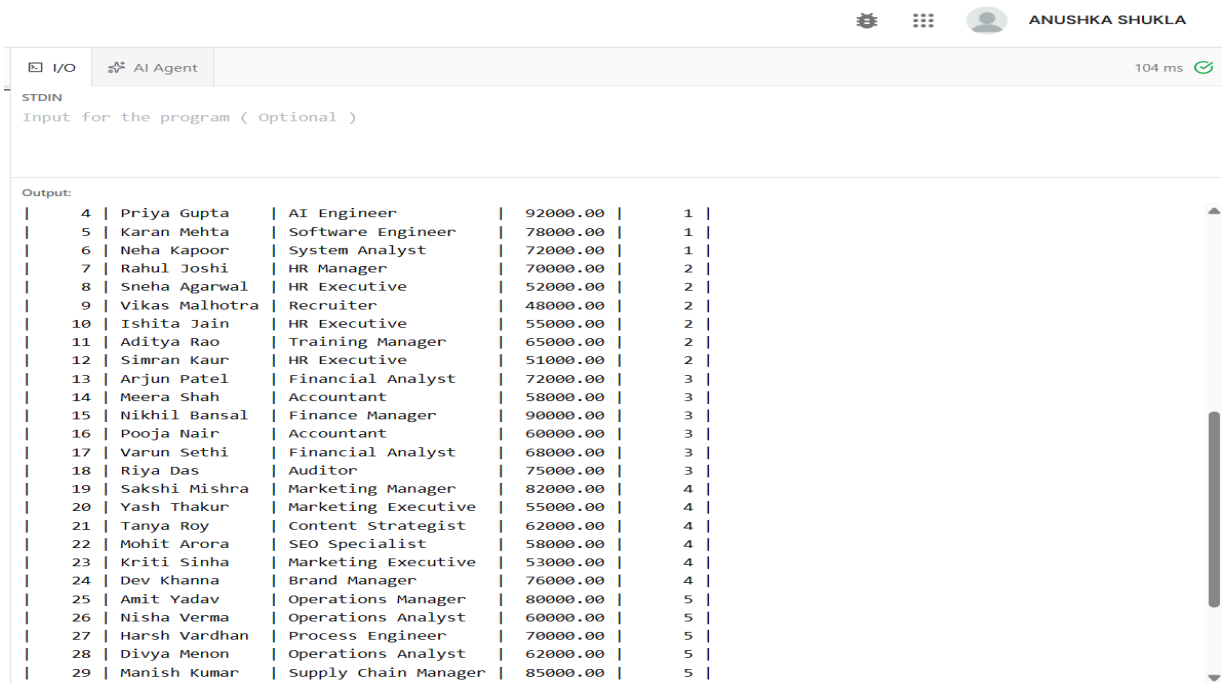
-- Negative salary:
-- UPDATE Employee
-- SET Salary = -5000
-- WHERE EmpID = 2;

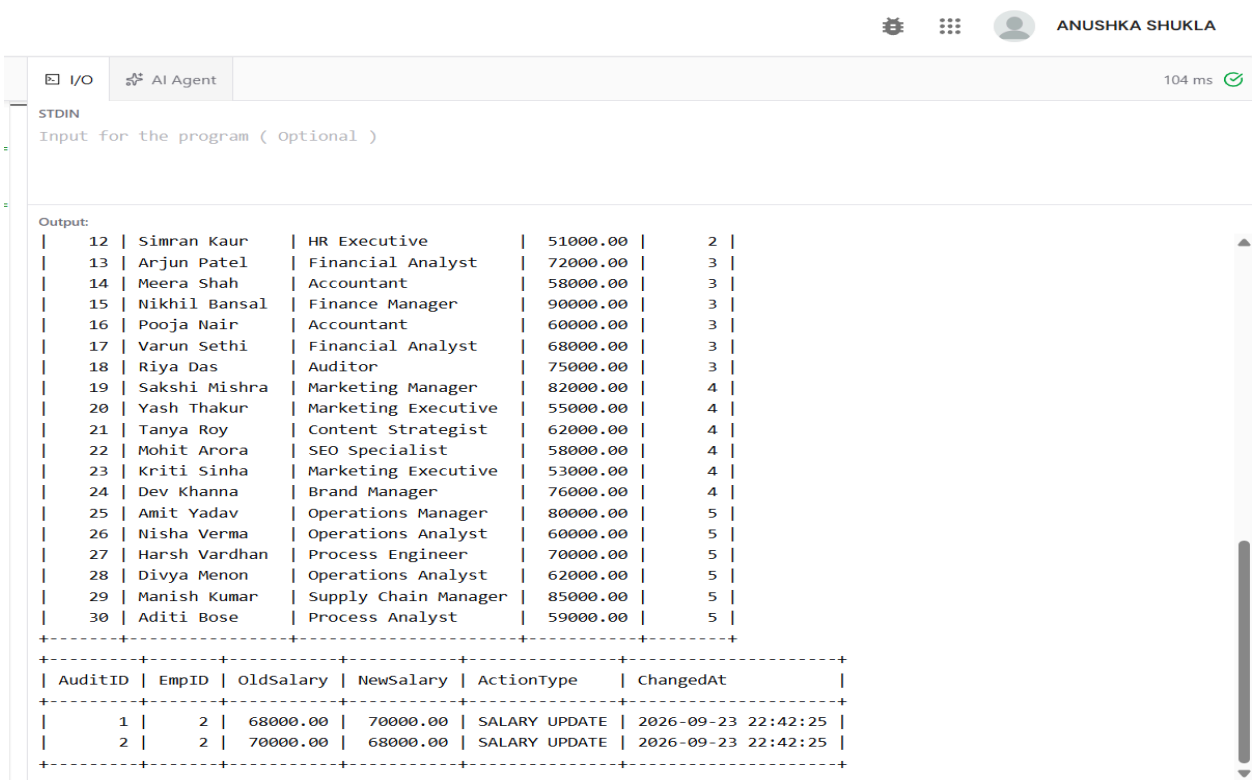
-- Zero salary:
-- UPDATE Employee
-- SET Salary = 0
-- WHERE EmpID = 2;

-- NULL salary:
-- UPDATE Employee
-- SET Salary = NULL
-- WHERE EmpID = 2;
```

OUTPUT


ANUSHKA SHUKLA
I/O
AI Agent
104 ms
STDIN
Input for the program (Optional)
Output:
+-------+--------------+--------+
| EmpID | EmpName | DeptID |
+-------+--------------+--------+
| 2 | Ananya Singh | 1 |
+-------+--------------+--------+
+-------+--------------+--------+
| EmpID | EmpName | DeptID |
+-------+--------------+--------+
| 2 | Ananya Singh | 2 |
+-------+--------------+--------+
+---------+-------+-----------+-----------+---------------+---------------------+
| AuditID | EmpID | OldSalary | NewSalary | ActionType | ChangedAt |
+---------+-------+-----------+-----------+---------------+---------------------+
| 1 | 2 | 68000.00 | 70000.00 | SALARY UPDATE | 2026-09-23 22:42:25 |
+---------+-------+-----------+-----------+---------------+---------------------+
+-------+---------------+----------+----------+------------+--------+-----------+
| EmpID | EmpName | JobTitle | Salary | HireDate | DeptID | ManagerID |
+-------+---------------+----------+----------+------------+--------+-----------+
| 31 | Test Employee | Tester | 50000.00 | 2026-01-01 | 2 | 7 |
+-------+---------------+----------+----------+------------+--------+-----------+
+-------+----------------+----------------------+-----------+--------+
| EmpID | EmpName | JobTitle | Salary | DeptID |
+-------+----------------+----------------------+-----------+--------+
1	Aarav Sharma	CEO	100000.00	1
2	Ananya Singh	Data Analyst	68000.00	2
3	Rohan Verma	IT Manager	85000.00	1


ANUSHKA SHUKLA
I/O
AI Agent
104 ms
STDIN
Input for the program (Optional)
Output:
4	Priya Gupta	AI Engineer	92000.00	1
5	Karan Mehta	Software Engineer	78000.00	1
6	Neha Kapoor	System Analyst	72000.00	1
7	Rahul Joshi	HR Manager	70000.00	2
8	Sneha Agarwal	HR Executive	52000.00	2
9	Vikas Malhotra	Recruiter	48000.00	2
10	Ishita Jain	HR Executive	55000.00	2
11	Aditya Rao	Training Manager	65000.00	2
12	Simran Kaur	HR Executive	51000.00	2
13	Arjun Patel	Financial Analyst	72000.00	3
14	Meera Shah	Accountant	58000.00	3
15	Nikhil Bansal	Finance Manager	90000.00	3
16	Pooja Nair	Accountant	60000.00	3
17	Varun Sethi	Financial Analyst	68000.00	3
18	Riya Das	Auditor	75000.00	3
19	Sakshi Mishra	Marketing Manager	82000.00	4
20	Yash Thakur	Marketing Executive	55000.00	4
21	Tanya Roy	Content Strategist	62000.00	4
22	Mohit Arora	SEO Specialist	58000.00	4
23	Kriti Sinha	Marketing Executive	53000.00	4
24	Dev Khanna	Brand Manager	76000.00	4
25	Amit Yadav	Operations Manager	80000.00	5
26	Nisha Verma	Operations Analyst	60000.00	5
27	Harsh Vardhan	Process Engineer	70000.00	5
28	Divya Menon	Operations Analyst	62000.00	5
29	Manish Kumar	Supply Chain Manager	85000.00	5



OBSERVATIONS/OUTCOME

1. The transfer_employee stored procedure was successfully created with input validation and error handling.
2. Valid employee transfers between departments were successfully performed.
3. Invalid employee and department IDs were rejected by the procedure.
4. Transfers to the employee's existing department were prevented.
5. The salary validation triggers successfully rejected zero, negative, and NULL salary values.
6. Valid salary modifications were successfully allowed.
7. The salary audit trigger automatically recorded salary changes in the SalaryAudit table.
8. Edge cases were tested to verify the reliability of the procedure and triggers.